

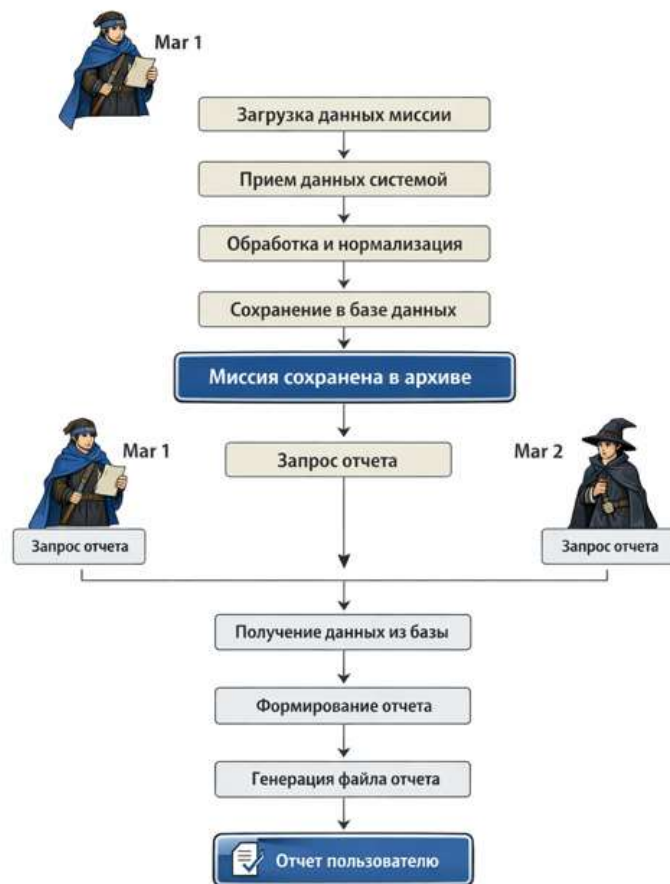
Лабораторная работа №3

Архив миссий Токийского магического колледжа .□

После инцидентов «Множественные протоколы» и «Сбой барьера» Совет старейшин окончательно утвердил новую архитектуру информационной системы колледжа.

Локальные анализаторы и временные серверы признаны нестабильными. Колледжу требуется полноценный **архивный веб- хаб**, способный не только обрабатывать миссии, но и хранить их, анализировать и предоставлять отчеты по запросу в любой момент времени. Вы назначены архитектором и разработчиком этой системы.

Ваша задача — разработать полноценное веб-приложение, обеспечивающее полный жизненный цикл миссий. Система должна функционировать как единый «архивный хаб памяти», доступный различным подразделениям колледжа.



Система должна уметь принимать описание миссии в разных форматах, как и раньше. Однако теперь результатом загрузки миссии не является автоматическое отображение разделов отчета в GUI. Вместо этого полученные данные должны быть приведены к единой внутренней модели и сохранены в базе данных, чтобы миссия стала частью долговременного архива колледжа.

После того как миссия сохранена, пользователь должен иметь возможность запросить генерацию отчета, указав конкретную миссию. В этом сценарии приложение

извлекает данные из базы данных, собирает всю информацию о миссии, участниках, проклятии и применениях техник и только затем формирует GUI-представление отчета.

На серверной стороне должна быть реализована логика:

- обработки входных данных
- нормализации моделей
- управления жизненным циклом миссий
- генерации отчетов
- обработки пользовательских запросов

Приложение должно предоставлять пользовательский интерфейс, который работает как единая точка взаимодействия с системой. Он позволяет:

- загружать миссии
- просматривать архив
- выбирать миссии
- формировать отчеты
- задавать параметры отчета

В отличие от предыдущей версии системы, теперь все миссии сохраняются в базе данных (PostgreSQL, или иная реляционная СУБД):

- обеспечивается долговременное хранение
- поддерживается повторный доступ к данным, повторные загрузки миссий и частичное обновление данных

Артефакты

Для допуска системы в эксплуатацию Совет старейшин требует не только работающий код.

Нужна “Карта архитектурного барьера”, показывающая устройство системы на разных уровнях, от пользователя до хранения и генерации документов. Поэтому в рамках лабораторной работы вы должны подготовить архитектурную схему в нотации C4, отражающую уровни системы, основные компоненты, связи между ними.

Перед вводом сервера в эксплуатацию Совет старейшин также требует представить «Свиток сетевого ритуала». Свиток должен позволять понять не структуру классов, а процесс взаимодействия компонентов. С точки зрения инженерии - это диаграмма последовательности, отражающая поток обработки HTTP-запроса. На свитке должно быть видно:

- кто инициирует обращение к серверу
- каким образом данные поступают в систему
- какие компоненты сервера участвуют в обработке
- на каком этапе происходит генерация документа
- что именно возвращается клиенту

Дополнительное испытание (для получения максимального балла)

1. SWAGGER для API проекта

Ваша задача состоит в том, чтобы оформить для архивного барьера полноценный HTTP API для работы с миссиями. Через него должно быть возможно передавать данные о миссиях, получать сведения из архива и вызывать формирование отчетов. Однако одного лишь набора маршрутов недостаточно. Любой маг, подключающийся к системе извне, должен понимать, какие запросы допустимы, какие данные следует передавать и какой ответ ожидать от барьера. Поэтому для получения дополнительных баллов требуется наложить на API печать автодокументации, а именно подключить Swagger и обеспечить автоматическое формирование описания всех доступных операций, чтобы карта внешнего взаимодействия с системой создавалась самой системой и всегда оставалась актуальной.

2. Unit-тесты для проекта

После создания внешних печатей система направляется в Палату контрольных прогонов, где ее проверяют не по красивым словам, а по реальному поведению. Для этого необходимо подготовить набор юнит тестов, покрывающих основную бизнес логику приложения. Испытания должны подтверждать, что миссии корректно принимаются, преобразуются и проходят внутреннюю обработку без искажений. Отдельное внимание следует уделить проверке правильности обработки входных данных, чтобы барьер не принимал поврежденные или некорректные сущности за подлинные архивные записи. Завершающим этапом становится проверка самого API.

В репозитории проекта должны сохраниться и сами тесты, и настроенная автодокументация, чтобы любой проверяющий мог убедиться, что система не является разовым ритуалом, а представляет собой полноценный инженерный артефакт.